



Q Type Code search



leightonw-cerebras

update the examples for SDK version 2.10.0

4866cf3 · last week

History

Name	Name	Last commit date
..		
README.rst	update the examples for SD...	3 years ago
code.csl	update the examples for SD...	last week
commands_wse2.sh	update the examples for SD...	2 years ago
histogram.csl	update the examples for SD...	last week
routes.csl	update the examples for SD...	last week
sdk-examples / benchmarks / histogram-torus /		Top

README.rst



HISTOGRAM

This example shows a CSL program that creates a histogram of a distributed input vector.

Histogram takes the following parameters:

- INPUT_SIZE is the number of input elements per PE.
- NUM_BUCKETS is the number of buckets present on each PE.

- `BUCKET_SIZE` is the size of each bucket.
- `HIST_WIDTH` is the width of the histogram kernel in PEs.
- `HIST_HEIGHT` is the height of the histogram kernel in PEs

Bucket Distribution

Each PE contains `NUM_BUCKETS` buckets of size `BUCKET_SIZE` each. The buckets are distributed statically across all the PEs in a row-major order. For example, given a kernel of size 4x4, a `BUCKET_SIZE` of 10, and `NUM_BUCKETS` = 5, we have the following bucket distribution:

[0, 50)	[50, 100)	[100, 150)	[150, 200)
-----	-----	-----	-----
[200, 250)	[250, 300)	[300, 350)	[350, 400)
-----	-----	-----	-----
[400, 450)	[450, 500)	[500, 550)	[550, 600)
-----	-----	-----	-----
[600, 650)	[650, 700)	[700, 750)	[750, 800)

Routing

The overall goal here is to implement point-to-point routing across the specified fabric region. We implement that using a 2 hop solution: sending data in the north/south direction, then actively reroute it in the east/west direction.

Each PE participates in 2 logical routes: one that travels in the north/south direction, and another that travels in the east/west direction. Each route is defined as a ring that spans all PEs north to south, or east to west.

Both routes are defined using 4 colors each: `c1`, `c2`, `c3`, and `c4`. On each route, even numbered PEs pass data in one direction, and odd numbered PEs pass it in the opposite direction. Each pair of colors alternates to pass data on one direction.

For example: On odd-numbered PEs '`c1`' and '`c2`' alternate to pass data east to west. On even-numbered PEs '`c3`' and '`c4`' alternate to pass data west to east.

The first and last PEs on each route are special: - PE 0 always receives on c4 and sends on c1, switching directions. - The last PE also switches directions, but the colors it uses depends on how many PEs there are in the ring.

Per-tile code

Each tile performs the following actions for each input data point:

1. Perform the required arithmetic to calculate the target PE for that data point.
2. Encode a wavelet that contains the target PE information.
3. If the target row ID is not the current row ID, send the wavelet in the north/south direction.
4. Otherwise, if the target column ID is different from the current column ID, send the wavelet in the east/west direction.
5. If both the target row and column IDs correspond to those of the current PE, increment the appropriate bucket count locally.

In the meantime, each PE handles traffic from both directions. Each wavelet is processed by each PE along the route, where it checks whether the wavelet belongs to it, in which case it keeps it, otherwise, it passes it along. Wavelets traveling in the north/south direction may need to be rerouted east/west, if the incoming wavelet matches the PE's row ID, but not column ID.

The wavelet encoding is as follows:

- 10 bits for the Y ID.
- 10 bits for the X ID.
- 12 bits for the bucket ID.

|---10(y)---|---10(x)---|----12(bucket)----|

Once the wavelet arrives at the target row (or if it already originated from the target row), the encoding changes to the following:

- 10 bits for the X ID.
- 10 empty bits.
- 12 bits for the bucket ID.

|---10(x)---|-----10-----|----12(bucket)-----|

This makes it easier to extract the X ID without performing any masking.

Termination criteria

No single PE knows when all the other PEs are done with all their work. Thus, we use a special column of PEs that collects completion signals from all the core histogram PEs. We call the row the `_tally_` kernel.

Each histogram PE keeps a count of how many wavelets it has put in a bucket. A "tally" PE at the end of each row polls and aggregates the counts. The last tally PE aggregates the counts from the other tally PEs. Once the count reaches the total count (the total number of inputs), it signals off the fabric.